

AI-powered semantic search in relational databases

using SQL Server 2025 and Ollama

Local embeddings, vector storage and semantic indexing on WideWorldImporters

Eva María Perales Belizón | CPIFP Alan Turing

eperbel453@g.educaand.es

From text search to semantic search

- Relational databases store much of the critical information used by organizations.
- Traditional searches depend on exact text matches.

With embeddings and vector indexes, we can search by meaning while preserving the relational model.

Project: semantic search with SQL Server 2025, Ollama and WideWorldImporters.

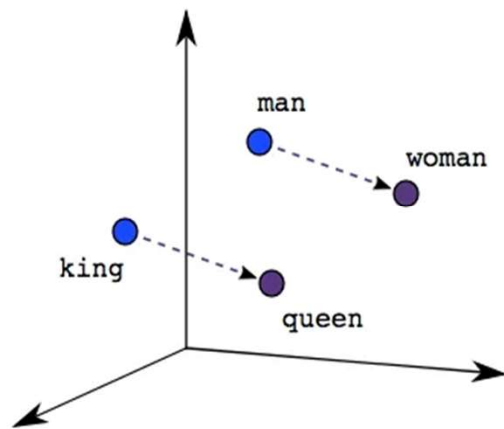
Key concepts: from text to vector

- Text search: finds word or pattern matches.
- Semantic search: searches by meaning, even when words do not match exactly.
- Embedding: a numerical vector representation of text, an image or another type of data.
- Vector: an ordered list of numbers; in this project, vector(768).

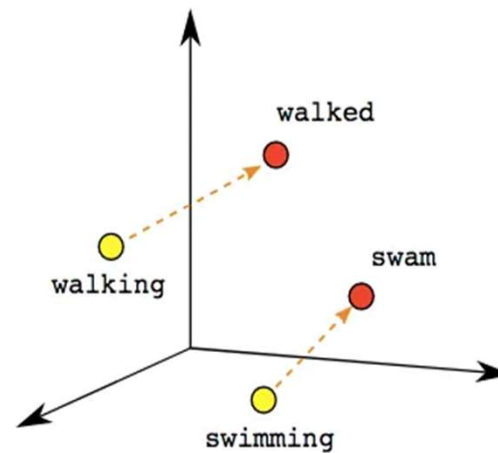
We convert products and queries into vectors using an AI model, then search for the nearest vectors to retrieve the most semantically related products.

Embeddings: vector representations of meaning

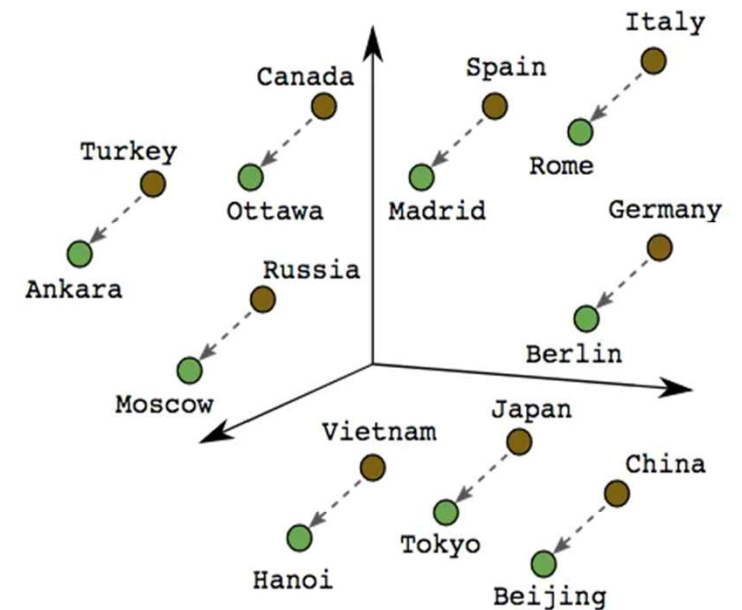
Items with similar meaning tend to be close to each other in vector space.



Male-Female



Verb Tense

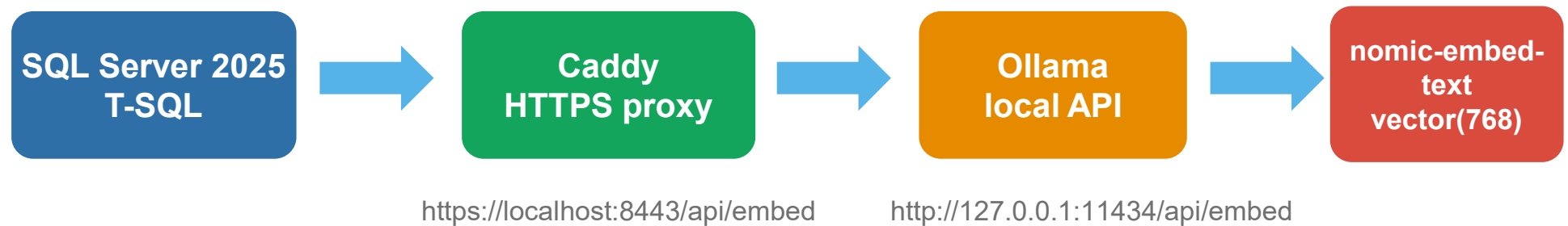


Country-Capital

Key concepts: similarity and efficiency

- **Ollama** Runs AI models locally, without relying on a cloud provider.
- **Embedding model** Converts text into embeddings. In this project: nomic-embed-text.
- **Cosine distance** Compares the orientation of two vectors.
Lower distance = higher similarity.
- **Vector index** Organizes embeddings to speed up similarity search.
- **DiskANN** Approximate algorithm for finding nearby vector neighbors.

Local project architecture



- **SQL Server needs an HTTPS endpoint to communicate with the external service.**
- **Caddy acts as an HTTPS bridge to Ollama, which runs locally over HTTP.**

Complete implementation flow



- The project does not replace the relational model: it adds a semantic layer on top of existing data.
- Structured data + embeddings + SQL queries = integrated semantic search.

SQL Server setup

```
ALTER DATABASE SCOPED CONFIGURATION  
SET PREVIEW_FEATURES = ON;
```

```
ALTER DATABASE WideWorldImporters  
SET COMPATIBILITY_LEVEL = 170;
```

```
sp_configure 'external rest endpoint  
enabled', 1;  
RECONFIGURE WITH OVERRIDE;
```

- **PREVIEW_FEATURES** enables preview vector features.
- **Compatibility level 170** aligns the database with **SQL Server 2025**.
- **external rest endpoint** allows calls to external services.

- **The credential registers the Caddy/Ollama HTTPS endpoint in a controlled way.**

Auxiliary table: dbo.ProductSemantic

```
CREATE TABLE dbo.ProductSemantic
(
    ProductSemanticID INT IDENTITY
                        PRIMARY KEY CLUSTERED,
    StockItemID INT NOT NULL,
    StockItemName NVARCHAR(100) NOT NULL,
    SemanticText NVARCHAR(MAX) NOT NULL,
    Embedding vector(768) NULL
);
```

- The original Warehouse.StockItems table is not modified.
- SemanticText contains the text that will be sent to the model.
- Embedding stores the vector generated for each product.
- StockItemID links each embedding back to its product in Warehouse.StockItems.

→ Intentional design: separate operational data from the semantic layer.

Registering the external model

```
CREATE EXTERNAL MODEL OllamaNomicEmbed  
WITH  
(  
  LOCATION = 'https://localhost:8443/api/embed',  
  API_FORMAT = 'Ollama',  
  MODEL_TYPE = EMBEDDINGS,  
  MODEL = 'nomic-embed-text'  
);
```

- **CREATE EXTERNAL MODEL** creates a database object.
- It registers where the model is located and how to communicate with it.
- It will then be used from T-SQL with **USE MODEL OllamaNomicEmbed**.

Generating embeddings inside SQL Server

```
UPDATE dbo.ProductSemantic
SET Embedding =
    CAST(
        AI_GENERATE_EMBEDDINGS(
            SemanticText
            USE MODEL OllamaNomicEmbed
        )
        AS vector(768)
    )
WHERE Embedding IS NULL;
```

- Takes SemanticText from each product.
- Calls Ollama through the external model.
- Obtains a numerical semantic representation.
- CAST AS vector(768) adapts the result to the column type.

Vector index: fast search by meaning

```
CREATE VECTOR INDEX  
IX_ProductSemantic_Embedding  
ON dbo.ProductSemantic (Embedding)  
WITH  
(  
    METRIC = 'cosine',  
    TYPE = 'DiskANN'  
);
```

→ The index is created after all embeddings have been generated.

→ cosine measures semantic closeness between text embeddings.

→ DiskANN enables approximate nearest-neighbor search.

→ Without an index: compare against every row.
With an index: locate nearby candidates efficiently.

Semantic search with VECTOR_SEARCH

```
SELECT
    ps.StockItemName AS ProductName
FROM VECTOR_SEARCH
(
    TABLE = dbo.ProductSemantic AS ps,
    COLUMN = Embedding,
    SIMILAR_TO = @queryEmbedding,
    METRIC = 'cosine',
    TOP_N = @top
) AS vs
WHERE CAST(1 - vs.distance AS DECIMAL(19,16)) >=
    @min_similarity
ORDER BY vs.distance ASC;
```

- **SIMILAR_TO**: vector generated from the search text.
- **METRIC**: same metric used by the vector index.
- **TOP_N**: number of nearby candidates.
- **WHERE**: applies a minimum similarity threshold.
- **ORDER BY**: ensures the most relevant results are shown first.

Three procedures, three levels of output

`find_relevant_product_names_vector_index`

Simple result

Returns only semantically related product names.

`find_relevant_products_vector_index`

Technical view

Shows SemanticText, distance and similarity.

`find_relevant_stock_items_vector_index`

Real-world case

Combines semantic search with business data.

Demo: natural-language queries

```
EXEC
dbo.find_relevant_stock_items_vector_index
    @prompt = N'usb gadget',
    @top = 10,
    @min_similarity = 0.3;
```

```
EXEC
dbo.find_relevant_stock_items_vector_index
    @prompt = N'funny IT-related gift',
    @top = 10,
    @min_similarity = 0.3;
```

- **Expected results: USB novelty drives, IT joke mugs, developer joke mugs.**
- **The search does not need an exact match: it searches for semantic closeness.**
- **The @min_similarity threshold helps control result quality.**
- **Relational filters remain available: color, price, category, supplier.**

Best practices and limitations

- The embedding depends on the quality of the text used in SemanticText.
- If the data is in English, English queries usually produce more stable results.
- Vector search can return close results, but not necessarily perfect ones.
- The solution improves when semantic search is combined with traditional SQL filters (BM25, VECTOR_SEARCH and RRF)
- It does not replace the relational model: it enriches it with a new way to search.

Not exclusive to SQL Server

SQL Server 2025 VECTOR, AI_GENERATE_EMBEDDINGS, VECTOR_SEARCH

Azure SQL Database vector capabilities in a managed service

PostgreSQL + pgvector vectors, distances and HNSW/IVFFlat indexes

Oracle Database integrated AI Vector Search

MySQL HeatWave vector store, embeddings and semantic search

The trend is clear: bringing AI and vector search into the core of relational databases.

Thank you for your attention

Questions or comments?

AI-powered semantic search with SQL Server 2025 and Ollama

